

TEMARL — What We Did, What We Found, and What It Means

Transformer-Enhanced Multi-Agent Reinforcement Learning for Adaptive Honeypot Deception. This report explains every decision, every experiment, every result table, and — honestly — why the architecture claim in the title did not hold, and what the thesis can claim instead.

The one-paragraph summary

We built a honeypot-deception system where four defender agents place decoys against an attacker whose behaviour comes from real attack data. **The system works:** it contains the attacker 60.4% of the time versus 54.8% for a defender with no attacker model, and 36.4% for no deception at all. But the specific claim in the title — that a **Transformer** reads attacker behaviour better than older sequence models — **is not true here**. We proved instead something more precise and still positive: the Transformer performs **statistically the same** as a GRU while using **44% of the parameters**.

0.6040

System success rate

+0.057

Gain over no-intent defender
($p=0.0004$)

44.4%

Transformer parameters vs
GRU

$p=0.0001$

Equivalence proven,
canonical map

1. What the thesis set out to prove

A honeypot is a fake computer used to trap attackers. If the defender can *guess what the attacker will do next*, it can put the right kind of fake bait in the right place. The thesis proposed that a **Transformer** — the model behind ChatGPT — would guess better than older sequence models such as a **GRU** or **LSTM**, because attacks are sequences of steps.

Two different claims live inside that sentence

Claim	What it needs	Status
A — System claim. Building this system (Transformer + multi-agent RL + MITRE-grounded deception) works.	Show it beats sensible defenders that have no attacker model.	SUPPORTED
B — Architecture claim. The Transformer beats GRU/LSTM at this job.	A fair head-to-head with statistics.	NOT SUPPORTED

Most of this report is about how we tested **B** as hard and as fairly as possible, and why the honest answer is "no" — while **A** survives and is what the thesis should be built on.

2. Full project history — every version, in order

The project ran through six numbered systems over its lifetime. Each one was kept so results stayed comparable. This section gives the results of every stage, including the two stages where the Transformer **did** come first.

Version	What it was	Outcome
v1	Original 10-node system, 310-dim state, 25-technique vocabulary, 5 actions	REJECTED — measurement was broken
v2	Rebuilt 5-zone environment, D3FEND payoff generated by rule	Worked. Transformer beat 3 of 5 baselines
v3	Variable-topology entity environment	Network-attention hypotheses null; history Transformer led
Phase 1	Line-by-line dataset audit	5 parser defects found and fixed
Phase 2	Six candidate explanations tested	2 refuted, 3 confirmed as our own artefacts
v4	Artefact-corrected comparison	8 nulls — later shown uninterpretable
v5	Repaired environment, 5 architectures	Valid test. Equivalence proven
v6	Powered replication of the best lead	Did not replicate; claim retired

2.1 v1 — why the original system was thrown away

The first system could not measure anything, and we rejected it on our own evidence before any comparison was published:

Problem found in v1	Measurement
All policies scored almost the same — the metric was saturated	DSR 0.90–0.97 for every policy
A defender with only a clock (no attacker model) beat the Transformer	step-counter accuracy 0.692
The attacker-intent vector was predictable from the state alone — it added nothing	$R^2 = 0.97\text{--}0.98$
Almost no room for intent to matter	headroom $\delta = 0.074$

This rejection is why the project has environment gates at all. v2 was built to fix each of these, and the headroom rose from 0.074 to **0.2698**.

2.2 v2 — the frozen control environment, and its results

Five pre-registered hypotheses (H1–H5), 10 seeds, 200 episodes each. **Training was imitation learning** against a profile oracle.

Method	Containment (DSR)	Decoy-match quality	What it is
Static-Null	0.3125	0.0500	No deception — the floor
Random Policy	0.4465	0.2508	Random decoys
Step-Counter Oracle	0.4880	0.3671	Perfect clock, <i>no</i> attacker model
Best-Fixed (decoy_persona)	0.5000	0.3560	One fixed decoy forever
NoTrans (h = 0)	0.5405	0.4631	Same model, intent vector switched off
Model B — GRU	0.6005	0.5944	Recurrent encoder
Model A — Transformer	0.6010	0.6004	The thesis model
Model C — SetEncoder	0.6070	0.6023	Order-blind control
Profile Oracle	0.6405	0.7380	Perfect intent — the ceiling

What v2 established

- **H1, H2, H3 SUPPORTED** — the Transformer beat the Step-Counter Oracle (+0.113), Best-Fixed (+0.101) and its own no-intent version (+0.061). **Intent modelling genuinely works.**
- **H4, H5 NOT SUPPORTED** — it did not beat the SetEncoder (0.6070) or the GRU (0.6005). The three encoders were within **0.0065** of each other.
- **The order-blind SetEncoder scored highest.** That was the first warning that sequence order carried little usable signal — two years of work before we understood why.

2.3 The encoder's own task — predicting the attacker's next move

Before being used for defence, each encoder was trained to predict the attacker's next technique. Here the Transformer **led on three of four measures**:

Model	Top-1	Top-3	Macro-F1	Perplexity (lower is better)
Transformer	0.1414	0.2930	0.1093	39.49
GRU	0.1312	0.2949	0.0967	46.33
SetEncoder	0.1144	0.2726	0.0877	46.04
<i>Majority-class baseline</i>	0.0595	—	—	80.00 (uniform)

2.4 Results on real honeypot data (COMISET)

Model	Top-1	Top-3	Macro-F1	Perplexity
Transformer	0.6463	0.9169	0.5112	2.684
GRU	0.6487	0.9095	0.4925	2.609
SetEncoder	0.4607	0.9060	0.4107	3.689
<i>Majority baseline</i>	0.3752	—	—	—
<i>Frequency baseline</i>	0.6835	—	—	—

Note the last row. A trivial frequency baseline (0.6835) beats *all three* neural models. COMISET is 98.5% repeated actions, so "predict the same thing again" is very hard to beat. This is why COMISET was never used for the main results.

2.5 CAM-LDS under three difficulty levels — and a genuine Transformer win

The same data can be split three ways, from hardest to easiest. This bracket is one of the most informative results in the project:

Protocol	What is held out	Transformer	GRU	SetEncoder	Majority
P1 — leave-one-campaign-out	An entire unseen campaign	0.0661	0.0461	0.0185	0.0224
P2 — within-campaign variant	A variant of a seen campaign	0.4934	0.6327	0.4836	0.0859
P3 — random run split	Random runs (easiest)	0.5454	0.6943	0.5487	0.1088

The Transformer wins the hardest protocol

On **P1** — predicting a campaign the model has *never seen* — the Transformer scores **0.0661** against the GRU's 0.0461 and the SetEncoder's 0.0185, against a majority baseline of 0.0224. It is the only model that clearly beats the baseline, improving on it by **+0.0477** versus the GRU's +0.0286 and the SetEncoder's +0.0010.

This is the pattern attention is supposed to show: an advantage that appears **when generalising to something unseen**, and disappears when memorisation is sufficient (P2, P3). It is reported here because it is real — but note the absolute numbers are tiny (6.6% accuracy), because with only 7 campaigns a held-out campaign shares almost nothing with the training data.

The **P2 gap of 0.1388** (GRU 0.6327 – Transformer 0.4934) became the headline "the GRU beats the Transformer" claim. **Phase 2 later proved that entire gap was our own configuration error** — see §2.8.

2.6 v3 — the variable-topology environment

Six combinations of history encoder × network encoder, 10 seeds:

History encoder	Network encoder	A	B	C	D	Train s
Transformer	DeepSets	0.4845	0.4595	0.4555	0.3830	70
Transformer	EntityTransformer	0.4685	0.4450	0.4540	0.3795	127
GRU	DeepSets	0.4625	0.4390	0.4450	0.3710	67
GRU	EntityTransformer	0.4575	0.4335	0.4315	0.3680	124
SetEncoder	DeepSets	0.4590	0.4400	0.4410	0.3635	65
SetEncoder	EntityTransformer	0.4415	0.4245	0.4230	0.3650	122

Something we under-reported at the time

The Transformer + DeepSets combination scored highest in all four conditions. The v3 report focused on hypotheses H6/H7/H8, which were about the *network* encoder (EntityTransformer vs DeepSets) and came out null — and in doing so it described the project as one where "the Transformer never won". That description was wrong, and this table is why.

The differences are small and were not individually significant, so this is not a claim of victory. But the statement "the Transformer never won" should never have been written, and it has been withdrawn.

2.7 Phase 1 — the dataset audit

Covered in full in §3. Summary: the grounding was rebuilt from the 36 source playbooks, a second independently written parser found three more defects, **8 lost attacker actions were recovered**, and three of our own published claims about the data were withdrawn.

2.8 Phase 2 — testing six explanations for why the Transformer was behind

Rather than accept the P2 result, we listed six possible causes and tested each. **Three turned out to be our own mistakes.**

#	Candidate explanation	Test	Verdict
2.1	The Transformer needs more data	Learning curves on both corpora	REFUTED — the gap <i>widens</i> with more data (−0.0187 per 10×)
2.2	The context window is too short	Lengths 8, 16, 32, 64	REFUTED — length 8 is best; longer is worse for every model
2.5	The position encoding is wrong	Sinusoidal vs Rotary vs None	CONFIRMED — our own error
2.4	The Transformer was never tuned	Equal 16-trial search for each model	CONFIRMED — our own error
2.6	It was trained on the wrong objective	Next-technique vs optimal-action target	CONFIRMED — our own error

Phase 2.5 — position encoding (CAM-LDS accuracy)

Context length	Transformer-Sinusoidal	Transformer-RoPE	Transformer-NoPE	GRU	RoPE advantage
8	0.8596	0.8856	0.8713	0.8856	+0.026
16	0.8375	0.8812	0.8553	0.8775	+0.044
32	0.8087	0.8678	0.8274	0.8748	+0.059
64	0.7961	0.8661	0.8043	0.8793	+0.070

Read the top row. With rotary position encoding at length 8, the Transformer scores **0.8856** and the GRU scores **0.8856** — a gap of **-0.00006**, i.e. a dead heat. The famous 0.1388 deficit was produced entirely by our choice of an unsuitable position encoding and untuned settings. The penalty from sinusoidal encoding also grows with length, from 2.6 points to 7.0 points.

Phase 2.4 — equal tuning budget

16 random-search trials per architecture, selected on validation data, scored on a held-out test set. Search space: 1–4 layers, 2–8 heads, feed-forward width 64–512, model width 32–128, four learning rates.

Stage	Transformer – GRU on CAM-LDS
Before (our original untuned configuration)	-0.1388 (d = 1.65, p = 5.7×10 ⁻¹⁴)
After equal tuning for both	+0.0000 — and at 44% of the parameters

Phase 2.6 — the training objective

The encoder had been trained to predict the attacker's next technique. But the defender does not need the technique — it needs the *best decoy*. Training directly on that target improved both models:

Model	Trained on next technique	Trained on best action	Gain
Transformer	0.867	0.934	+0.067
GRU	0.844	0.921	+0.077

The lesson from Phase 2 is uncomfortable and worth stating plainly. The single largest "finding" in the project's first two years — that recurrence beats attention on this data by a large, highly significant margin — was **an artefact of our own experimental setup**. It survived because nobody tested the alternative explanations. Phase 2 is the reason the later pre-registrations exist.

3. The dataset — what it is and everything we did to it

3.1 Which dataset, and why

CAM-LDS is a published set of real, scripted attack campaigns run on a test network (the "AttackBed" testbed). Each campaign is a sequence of MITRE ATT&CK techniques — the standard catalogue of what attackers actually do (for example **T1059** = running malicious commands).

We also examined **COMISET**, a much larger honeypot log dataset, but used it only for encoder pre-training experiments — never for the main results.

Property	Value	Meaning in plain words
Playbooks (runs)	36	36 recorded attack runs
Labelled technique steps	1,347	1,347 individual attacker actions
Distinct campaigns (scenarios)	7	7 different attack "storylines"
Parent techniques	82	82 kinds of attacker action
Tactics	13	13 attacker goals
Kish effective sample size	3.09	The 36 runs are really only ~3 independent campaigns. This is a very small dataset.

3.2 We rebuilt the dataset from source, and found five real bugs

The original version was reconstructed by hand from the paper's text. We replaced it by parsing the 36 published playbook files directly. Then a **second, independently written parser** was used to check the first. It found bugs the first one had.

Round	Defect	Damage
1	Two files use singular <code>tactic:</code> instead of <code>tactics:</code>	An entire campaign silently dropped
1	<code>.j2</code> and <code>.j2.yml</code> are the same playbook	One campaign counted twice
2	Block-order bug — when <code>tactics:</code> appeared before <code>techniques:</code> , nothing was recorded	Lost T1190, T1059.004, T1095
2	Inline # comment bug (×2 variants)	Lost 1× T1222.002 and 4× T1219
Net effect of round 2		8 lost actions recovered: 1,339 → 1,347

Separately, source verification found **6 technique types the hand reconstruction had missed entirely** (T1069, T1218, T1518, T1572, T1614, T1615) — **6.20% of all attacker actions** were being silently thrown away as "unknown". We extended the vocabulary from 78 to 84 techniques, *append-only* so old IDs stayed valid.

Three claims we had to withdraw about our own data

- "82 techniques and 13 tactics match the published paper exactly."** FALSE. The apparent perfect match was an accident of the bugs cancelling out. It is 82 against the paper's 81.
- "The hand reconstruction invented technique T1095."** FALSE. T1095 is genuinely there in `scenario_7.j2` line 95 — the block-order bug was hiding it. The reconstruction's precision is **100%**, not 98.7%.
- "COMISET is 1156× bigger than CAM-LDS."** MISLEADING. 98.5% of its steps are the same action repeated. After collapsing repeats it has 23,754 usable transitions — the real advantage is **about 18×**, not 1156×.

3.3 Yes — we clustered the dataset, and that clustering caused the failure

You asked specifically about clustering. We did cluster, in one place, and it turned out to be the single most damaging decision in the project.

The 7 campaigns were **grouped into 3 "intent classes"** by a function called `consolidate_intent_classes()`. The grouping rule was: **campaigns that need the same best decoy are the same intent**. The three clusters were even *named after their own answers*: `decoy_persona`, `decoy_file`, `decoy_credential`.

Why this was fatal for the thesis question

If you group attacks by "which answer is correct", then knowing the group **is** the answer. The defender never has to read the sequence — it only has to work out which of 3 boxes it is in.

We measured exactly how bad this was. Within each cluster, the best decoy was the same for **84 out of 84 techniques** in two clusters, and 83 of 84 in the third. So:

What the defender knows	Expected payoff	Gain
Nothing (best single fixed decoy)	0.3584	—
The cluster label (3-way guess)	0.7426	+0.3842
Perfect prediction of the next technique	0.7427	+0.0001

Perfect sequence prediction was worth +0.0001 — that is 0.0% of what the cluster label was worth.

The task was 3-way classification wearing a sequence costume. No Transformer, GRU or LSTM could possibly have distinguished itself on it.

3.4 The estimator bug that destroyed 87% of the sequence structure

The environment turns each campaign into a **transition table**: "after technique A, how likely is technique B?" The code that built this table had one damaging line:

`T[:, j] += 1.2` — added identically to **every row**.

A number added identically to every row carries **zero information** about what follows what — by definition. Worse, it added 7.2–8.4 units of mass per row against only 4.0 units from the real observed data, so the noise outweighed the signal roughly 2:1. Between 55% and 74% of rows became literally identical to one another.

Measure	Before	After removing the line
Sequence information kept, per campaign	0.627 bits	1.935 bits
(the raw data actually contains)		4.83 bits — 85.4% predictable
Value of perfect sequence prediction (payoff)	+0.0002	+0.1776
Value of perfect sequence prediction (success rate)	+0.0005	+0.0800
Different best decoys per campaign	1.33	4.33

Proof this was a one-line change, not tuning. Setting the constant back to 1.2 reproduces the old campaign profiles **bit-for-bit** (largest difference 0.000000). We also tested two *other* repairs we expected to help — better smoothing, and using all 7 campaigns instead of 3 — and **rejected both**, because measurement showed they made things *worse* (−0.0588 and −0.0305). Only the change justified by the mathematics was kept.

4. The environments — three of them, and why each exists

An "environment" is the simulated network where the attacker and the four defender agents interact. We built three, each kept alongside the last so results stay comparable.

Environment	What it is	Why we built it
1. <code>env_v2.py</code> The frozen control	Fixed 5-zone network. 4 defender agents, one per defended zone. Attacker moves between zones emitting ATT&CK techniques. Reward = $0.30 \times \text{containment} + 0.70 \times \text{decoy-match quality}$.	The scientific control. Never modified. Everything else must reproduce it exactly.
2. <code>env_entity.py</code> Variable topology	Same rules, but the network can have any number of zones and hosts. The defender sees a <i>list of machines</i> with features, not a fixed vector. Actions become a pair: (<i>what decoy, on which machine</i>).	To test whether attention helps when the network shape changes — following Symes Thompson et al. (arXiv:2410.17647). Verified to reproduce env_v2 exactly: 0 mismatches over 1,126 steps.
3. <code>profiles_v5.py</code> + calibration The repaired environment	Same environment, but the attacker model is rebuilt without the damaging constant, and the capture dynamics are re-tuned so the metric still discriminates.	Because we discovered environments 1–2 could not answer the thesis question at all (see §2.3–2.4).

4.1 The four test conditions ("regimes")

Every model is tested on four network types — this is how we test generalisation:

Regime	What it is	Tests
A	The original 5-zone map	Basic competence
B	New maps, sizes seen in training	Normal generalisation
C	Unseen structure (different firewall density)	Structural generalisation
D	Unseen sizes (9, 11, 13 zones)	Size generalisation — the hardest

4.2 Repairing the environment broke the measurement, and we had to fix that too

Removing the damaging constant made episodes **3× longer** (5.45 → 18–22 steps). That caused the success rate to **saturate at 0.79–0.98** — every policy looked good, so nothing could be compared. This is exactly the failure the project's own environment gate was written to catch (version 1 of this project was rejected for it).

So we re-calibrated two constants against **four criteria fixed in advance**, using only non-learning baseline policies (so no architecture could influence the choice). Of 20 combinations tested, **exactly one passed all four**:

Criterion (declared before testing)	Requirement	Selected setting
C1 — not saturated	no-deception policy between 0.25 and 0.40	0.367 ✓
C2 — discriminating	best – worst ≥ 0.15	0.217 ✓
C3 — history worth reading	episode ≥ 10 steps	18.4 ✓
C4 — sequence actually pays	sequence value ≥ 0.03	0.050 ✓
Chosen configuration		<code>goal_steps=3, cap_scale=0.35</code>

5. The models we built and how much we changed them

5.1 Six encoders in total

Two *kinds* of encoder are involved. A **history encoder** reads the attacker's past techniques. A **network encoder** reads the list of machines.

Model	Type	Parameters	Role and what we changed
Transformer-RoPE	History	156,310	The thesis's model. We changed it twice: (1) replaced sinusoidal position encoding with rotary (RoPE) ; (2) re-tuned depth/width with an equal search budget. Both changes <i>helped</i> it.
Transformer-Sinusoidal	History	161,174	The original version, kept to measure how much our own configuration was hurting it.
GRU	History	351,702	The main recurrent competitor. We added <code>pack_padded_sequence</code> so padding cannot contaminate it — without this the GRU is unfairly handicapped and the Transformer "wins" for a fake reason.
GRU-d32	History	236,598	A control we invented: a GRU with the Transformer's narrower output width, to test whether any Transformer advantage was really just about output size.
LSTM	History	450,518	Added at your request. The original manuscript listed it as future work and it was never built. Same padding discipline; given the GRU's tuned settings verbatim.
SetEncoder	History	~74,000	Order-blind control. If the Transformer cannot beat this, sequence order carries no usable signal.
EntityTransformer	Network	74,369	Attention over machines. Capacity-matched to DeepSets within 0.7%.
DeepSets	Network	73,828	Order-blind pooling over machines.

Fairness rules we enforced on every model

- **Capacity matched** where compared (entity encoders within 0.7%).
- **Identical padding handling** — the single most common way to fake a Transformer win is to let padding leak into the RNN.
- **Equal tuning budget** — 16 random-search trials each, selected on validation, scored on a held-out test set.
- **Identical data, identical seeds, identical evaluation** — every comparison is *paired* on the same random seeds.

5.2 The most important model finding: we had been handicapping our own Transformer

An earlier result said the GRU beat the Transformer on attack prediction by **+0.1388** (a very large, highly significant gap). We investigated and found that gap was **our own fault**, not a property of the models:

Fix applied equally to both models	Effect on the gap
Rotary position encoding instead of sinusoidal	+0.026 to +0.070
Equal 16-trial hyperparameter search	closes the rest
Corrected gap	+0.0000 — a tie, at 44% of the parameters

6. How we benchmarked — the measuring instrument

This is the part that turned out to matter most, and it is a genuine contribution of the thesis.

6.1 The reference ladder

A number like "success rate 0.45" means nothing on its own. So we built a **ladder of reference policies**, each given more information than the last:

Rung	What it knows
null	Nothing — places no decoys. The floor.
random	Picks decoys at random.
best_fixed	Always plays the single best decoy. No attacker model at all — this is the bar a learned model must clear.
profile	Perfectly knows which of the 3 campaign clusters it faces.
transition	Perfectly predicts the next technique. This is the ceiling for any history encoder — it is the encoder's whole job, done perfectly and for free.
clairvoyant	Sees the actual next technique. Unattainable; bounds everything above.

The key quantity: "sequence value"

sequence value = (perfect next-technique model) – (perfect cluster label)

If this is zero, **no sequence model can possibly distinguish itself**, however good it is. This single number is what revealed that the original experiment was unable to answer its own question.

6.2 The validity gate

Before comparing two models, we now check that **each one actually learned the task** — it must beat `best_fixed` significantly. Without this check, a "no difference" result is meaningless, because it cannot tell apart "both are good" from "neither learned anything".

7. Results — every experiment, in order

7.1 The measuring instrument, before and after repair

Policy	A	B	C	D	A	B	C	D
	ORIGINAL environment				REPAIRED environment			
null (no deception)	0.3150	0.3090	0.3115	0.3030	0.3680	0.3635	0.3615	0.3515
random	0.4235	0.4090	0.3990	0.3495	0.5320	0.4935	0.5030	0.4080
best_fixed (no attacker model)	0.4540	0.4340	0.4340	0.3625	0.5920	0.5475	0.5470	0.4465
profile (perfect cluster)	0.6245	0.5820	0.5825	0.4470	0.6050	0.5590	0.5565	0.4490
transition (perfect prediction)	0.6250	0.5825	0.5830	0.4475	0.6845	0.6390	0.6370	0.5035
clairvoyant (upper bound)	0.6610	0.6200	0.6180	0.4800	0.7430	0.7155	0.7150	0.5730
SEQUENCE VALUE	+0.0005	+0.0005	+0.0005	+0.0005	+0.0795	+0.0800	+0.0805	+0.0545
Episode length (steps)	5.5	5.6	5.6	5.8	17.3	18.1	18.2	20.2

Read the bold row. In the original environment, a **perfect** sequence model was worth **+0.0005** — nothing. After repair it is worth **+0.0800**, a **160× increase**. Only the second column is a fair test of the thesis.

7.2 Experiment v4 — on the original environment (the uninterpretable one)

Model / objective	A	B	C	D	Params	Train s
Transformer-RoPE / action	0.4525	0.4300	0.4295	0.3655	156,310	120
Transformer-RoPE / next	0.4695	0.4410	0.4415	0.3680	156,310	119
Transformer-Sin / action	0.4585	0.4320	0.4360	0.3575	161,174	117
Transformer-Sin / next	0.4435	0.4220	0.4240	0.3655	161,174	116
GRU / action	0.4725	0.4500	0.4530	0.3805	351,702	76
GRU / next	0.4545	0.4345	0.4355	0.3660	351,702	77

All 8 statistical comparisons came out non-significant. **But this result is worthless**, for two reasons we discovered afterwards:

Why v4 could not answer anything — the validity gate it never ran

Arm	vs best_fixed	Headroom used	p	Learned?
Transformer-RoPE / action	−0.0040	−2.7%	0.852	NO
Transformer-RoPE / next	+0.0070	4.7%	0.497	NO
Transformer-Sin / action	−0.0020	−1.4%	0.881	NO
Transformer-Sin / next	−0.0120	−8.1%	0.446	NO
GRU / action	+0.0160	10.8%	0.050	yes
GRU / next	+0.0005	0.3%	0.967	NO

Five of six models never beat "always play one fixed decoy". Comparing two models that never learned tells you nothing about their architectures. This is why we rebuilt the experiment.

7.3 Experiment v5 — on the repaired environment (the real test)

Validity gate first — this time all five models genuinely learned:

Model	Success (B)	vs best_fixed	Headroom used	p	Gate
Transformer-RoPE / DeepSets	0.6040	+0.0565	61.7%	0.0004	PASS
GRU / DeepSets	0.6100	+0.0625	68.3%	0.0000	PASS
GRU-d32 / DeepSets	0.6055	+0.0580	63.4%	0.0018	PASS
Transformer-RoPE / EntityTransformer	0.6075	+0.0600	65.6%	0.0001	PASS
GRU / EntityTransformer	0.6175	+0.0700	76.5%	0.0001	PASS
LSTM / DeepSets added later	0.6085	+0.0610	66.7%	0.0001	PASS

Full results, all four regimes (10 paired seeds, 200 test episodes each):

Model	A	B	C	D	Params	Train s	Teacher match
Transformer-RoPE / DeepSets	0.6540	0.6040	0.6025	0.4865	156,310	115	0.773
GRU / DeepSets	0.6550	0.6100	0.6080	0.4820	351,702	51	0.811
GRU-d32 / DeepSets	0.6520	0.6055	0.6050	0.4920	236,598	48	0.754
Transformer-RoPE / EntityTransformer	0.6525	0.6075	0.6070	0.4810	156,310	126	0.771
GRU / EntityTransformer	0.6565	0.6175	0.6135	0.5000	351,702	64	0.807
LSTM / DeepSets added later	0.6560	0.6085	0.6115	0.4845	450,518	54	0.801

Six different architectures span just **0.0045** on map A and **0.019** on map D. They are, for practical purposes, the same.

7.4 The LSTM comparison — completing the ablation

The original manuscript listed an LSTM comparison as future work and it was never built, so every "attention versus recurrence" statement had been tested against only **one** recurrent model. We added the LSTM with the identical padding discipline and the GRU's tuned settings.

Regime	Transformer	LSTM	Difference	95% CI	p (difference)	p (equivalence)	Verdict
A canonical	0.6540	0.6560	−0.0020	[−0.012, +0.008]	0.699	0.0029	EQUIVALENT
B trained sizes	0.6040	0.6085	−0.0045	[−0.016, +0.007]	0.450	0.0118	EQUIVALENT
C unseen structure	0.6025	0.6115	−0.0090	[−0.020, +0.002]	0.150	0.0432	EQUIVALENT
D unseen sizes	0.4865	0.4845	+0.0020	[−0.015, +0.019]	0.817	0.0304	EQUIVALENT

Against the LSTM, equivalence is proven in all four conditions — even stronger than against the GRU, where regime D could only be bounded. And the LSTM is the largest model of the three:

Model	Parameters	Relative size	Training time
Transformer-RoPE	156,310	1.00×	115 s
GRU	351,702	2.25×	51 s
LSTM	450,518	2.88×	54 s

The Transformer matches both recurrent models using **44.4% of the GRU's parameters and 34.7% of the LSTM's**. It is, however, **2× slower to train** — the honest trade-off.

7.5 The eight pre-registered statistical tests

All hypotheses were written down and committed to version control **before** the experiment ran (commit 1413afb), so they could not be adjusted afterwards.

Test	What it asks	Difference	95% CI	p (Holm)	Result
H16-D	Transformer > GRU, unseen sizes	+0.0045	[−0.012, +0.021]	1.000	No
H16-C	Transformer > GRU, unseen structure	−0.0055	[−0.013, +0.002]	1.000	No
H17-B	Transformer vs GRU, trained sizes	−0.0060	[−0.016, +0.004]	1.000	No
H17-A	Transformer vs GRU, canonical	−0.0010	[−0.007, +0.005]	1.000	No
H18-D	Transformer > width-matched GRU	−0.0055	[−0.023, +0.012]	1.000	No
H18-B	same, trained sizes	−0.0015	[−0.018, +0.015]	1.000	No
H19-DT	Entity attention helps Transformer	−0.0055	[−0.024, +0.013]	1.000	No
H19-DG	Entity attention helps GRU	+0.0180	[−0.000, +0.036]	0.690	Closest

7.6 Experiment v6 — we gave the one promising result a proper second chance

H19-DG was the only test whose confidence interval nearly cleared zero. With 10 seeds we could only detect very large effects, so this was **underpowered, not disproved**. We calculated the sample size needed and **pre-registered a replication at exactly 31 seeds** (commit 7261a4a), using **fresh seeds** so it was independent.

Run	n	EntityTransformer	DeepSets	Difference	Effect size	p
v5 original	10	0.5000	0.4820	+0.0180	+0.61	—
v6 replication	31	0.4597	0.4652	-0.0055	-0.17	0.826
Pooled	41	0.4695	0.4693	+0.0002	+0.01	0.481

The replication came out in the **opposite direction**. Pooled over all 41 seeds the difference is **+0.0002** — essentially exactly zero. As promised in advance, this claim is retired and no further seeds were added.

8. How we know the architecture claim fails — four independent ways

#	Evidence	Result
1	8 pre-registered tests, corrected for multiple comparisons, on a validated environment	All non-significant
2	Properly powered replication (n=41 pooled) of the best lead	+0.0002
3	Equivalence testing — proving the difference is <i>below a threshold</i>	Equivalence proven, p=0.0001
4	Every small-sample "win" retested at proper sample size	All four vanished

Four times a promising result disappeared when tested properly		
Apparent finding	At small n	At proper n
Transformer wins at unseen sizes	+0.0333 (n=3)	+0.0045 (n=10)
Width control confirms it is architectural	27.6% (n=3)	79.8% (n=10)
Entity attention helps the GRU	d = 0.61 (n=10)	d = -0.17 (n=31)
Pooled final estimate	—	+0.0002 (n=41)
Seed-to-seed variation (SD ≈ 0.068) was larger than every effect ever claimed from small samples. This is itself a reportable finding.		

9. Why it failed — the five real reasons

#	Reason	Evidence
1	The dataset is tiny. 7 campaigns, effective sample size 3.09. Transformers need large data.	Kish n = 3.09
2	The sequences are short. 18–22 steps, 16-token context. Attention's advantage comes from long-range dependencies that do not exist here.	Episode length 18.4
3	The clustering removed the sequence problem. Grouping campaigns by their own answer made it 3-way classification.	Sequence value +0.0001
4	The estimator discarded 87% of the sequence structure.	0.627 of 4.83 bits
5	The task is nearly saturated. Both models already capture 62–77% of everything a perfect predictor could achieve, leaving ~0.02–0.03 for any improvement — smaller than random seed noise.	Headroom table §6.3

Important nuance for your defence: reasons 1, 2 and 5 are properties of the *problem*, not faults in the Transformer. The model is not broken. The task simply does not have enough sequential structure, or enough remaining headroom, for attention to demonstrate an advantage.

10. What the thesis CAN claim — the positive results

Claim 1 — The system works (this is the main contribution)

Defender	Containment rate	Meaning
No deception	0.3635	Baseline
Random decoys	0.4935	Decoys help
Best fixed decoy, no attacker model	0.5475	Smart but blind
TEMARL (learned attacker model)	0.6040 – 0.6175	+0.057 to +0.070, p ≤ 0.0018
Perfect attacker prediction (ceiling)	0.6390	Upper limit

TEMARL captures **62–77%** of everything a perfect attacker-prediction system could achieve. **Attacker-intent modelling is worth +0.148 to +0.171** containment — this validates the entire premise of the architecture.

Claim 2 — Proven equivalence at 44% of the parameters

This is not "we failed to find a difference" — it is a statistical proof that the difference is **smaller than a stated threshold** (TOST test, 0.02 margin).

Regime	Difference	p (equivalence)	Conclusion
A canonical	−0.0010	0.0001	EQUIVALENT
B trained sizes	−0.0060	0.0124	EQUIVALENT
C unseen structure	−0.0055	0.0029	EQUIVALENT
D unseen sizes	+0.0045	0.0527	bounded below 0.020

Against the **LSTM** the result is stronger still — equivalence proven in **all four** conditions ($p = 0.0029, 0.0118, 0.0432, 0.0304$).

Transformer 156,310 parameters vs GRU 351,702 (44.4%) and vs LSTM 450,518 (34.7%). Equal deception performance for roughly a third of the model. The cost is 2x training time.

Claim 3 — Methodological findings (publishable in their own right)

- **A pre-registered environment gate can pass while the environment is useless for the actual hypothesis.** Ours did: it asked "is attacker information worth anything?" (yes, 0.2698) and never asked "is *sequence* information worth anything beyond a class label?" (+0.0001).
- **Architecture comparisons need a baseline ladder and a validity gate.** Without them, v4 produced 8 null results from 5 models that had never learned.
- **Small-sample effects in this domain do not survive.** Four for four.

Claim 4 — Data contributions

- A source-verified CAM-LDS grounding with **five parser defects found and fixed**, verified by an independently written second parser.
- **A quantified estimator defect:** a column-constant in a transition matrix that destroyed 87% of the corpus's sequential information.
- Vocabulary extension recovering **6.20% of attacker actions** that were being discarded as unknown.

11. Honest limitations (state these before an examiner finds them)

1. **Training is behaviour cloning, not reinforcement learning.** Under PPO no model beat the memoryless baseline at any budget tried (240 episodes $\rightarrow$ 0.510, 1,200 $\rightarrow$ 0.523, baseline 0.5475). We report encoder quality under matched supervision, not a full MARL result.
2. **Why PPO fails is unresolved.** Two hypotheses were tested and both were false: reward/metric conflict (correlation was **+0.983**, so no conflict), and a log-probability averaging bug (fixing it made results *worse*).
3. **Sample size $n=10$** detects only very large effects; the confidence intervals, not the p-values, carry the argument.

4. **One environment, one scripted attacker.** A learning attacker could change everything.
5. **The repaired environment is calibrated**, chosen from 20 candidates against pre-declared criteria — a different calibration might differ.
6. **The LSTM was given the GRU's tuned settings**, not its own search.

12. Timeline — every major decision

Stage	Decision	Why	Outcome
Data	Re-parse CAM-LDS from source playbooks	Hand reconstruction was unverified	6 missing techniques found (6.20%)
Data	Write a second independent parser	To check the first	3 more bugs, 8 actions recovered
Model	Add rotary position encoding	Sinusoidal was handicapping the Transformer	+0.026 to +0.070
Model	Equal 16-trial tuning for both	Fairness	Gap 0.1388 → 0.0000
Method	Build the reference ladder	Numbers were uninterpretable	Found sequence value = +0.0005
Env	Remove the transition-matrix constant	It carried zero information by definition	Sequence value ×160
Env	Recalibrate capture dynamics	Repair had saturated the metric	1 of 20 settings passed all gates
Method	Switch PPO → behaviour cloning	No model beat baseline under PPO (<i>decided using GRU only</i>)	All 5 models then learned
Stats	Add equivalence testing	"Not significant" understated the finding	Equivalence proven, p=0.0001
Stats	Power-analysed replication at n=31	Best lead was underpowered	Did not replicate; retired
Model	Add LSTM	Ablation was incomplete	See §6

13. Integrity record — what an examiner will check

Commit	Content	Timing
7499d29	v4 pre-registration	Before the v4 run
1413afb	v5 pre-registration + environment repair	Before the v5 run
0c25e12	v5 result, all 8 contrasts reported	After
7261a4a	Equivalence tests + v6 pre-registration	Before the v6 run
4b53a26	v6 replication result	After

One number that must never be quoted

The v5 data, re-tested with a one-sided test, gives $p = 0.0431$, which looks significant. **It is not a valid result.** v5 was registered as a two-sided, multiplicity-corrected test and stands at $p = 0.690$. Switching test type after seeing the data is exactly the manipulation that invalidates a thesis. It is recorded here so nobody quotes it by accident.

TEMARL · BRAC University · Apurba Roy (23101012), Tushit Roy (23101350), Mark Dipto Gomes (23101537), Farjana Akter Shumi (23101059) · Supervisor: Md. Sakir Hossain · Report generated 8 September 2026. All figures traceable to `results_v4/`, `results_v5/`, `results_v6/`, `results_v7/` in the repository.